

ASSIGNMENT 1

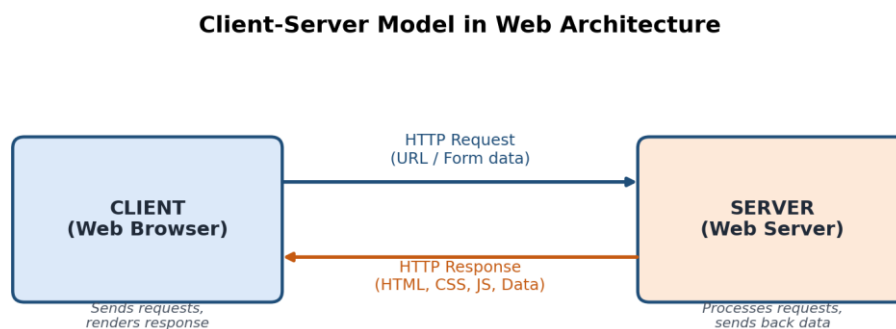
Q1. Difference between Frontend, Backend, and Full-Stack Development

Frontend is everything the user sees and clicks on. The layout, the colors, the buttons, forms, little animations when you hover over something, all of it. It's built with HTML for the structure, CSS for how it looks, and JavaScript to make it interactive, and nowadays people usually add a framework like React or Vue on top instead of doing it all by hand. A simple example: on Amazon's homepage, the product images, the filter sidebar on the left, the way the “Add to Cart” button changes color when you hover over it.

Backend is the part you never actually see. It lives on the server and takes care of things like checking whether an item is still in stock, working out the total price, checking login details, and reading/writing to the database. Usually written in something like Node.js, Python (Django or Flask), Java Spring, or PHP. So, when you place an order on Amazon, it's the backend quietly checking stock, running the payment, and updating the records, none of which you actually see happen.

Full stack just means someone who can do both. They build the interface and also the server logic and database behind it. Say someone's building a small food-delivery app on their own — they'd design the menu screens themselves (that's the frontend part) and also write the backend code that keeps track of orders and restaurant listings.

Q2. Diagram: How the Client-Server Model Works in Web Architecture



Q3. How a Browser Requests and Displays a Web Page from a Web Server

- The browser first needs to know where to send the request, so it looks up the domain name through DNS and turns it into an IP address (something like `www.example.com` becomes `93.184.216.34`).
- Once it has that IP, it opens a TCP connection with the server. If it's an HTTPS site, a TLS handshake happens around here too, so the connection is encrypted.
- Then the browser sends an HTTP GET request asking for a specific page or resource.
- The web server on the other end picks it up and figures out what's actually being asked for. Sometimes it's just a plain HTML file sitting there; other times it must run backend code and pull something from a database first. Either way, it puts together a response.
- That response makes its way back to the browser with a status code attached (200 OK usually, 404 if the page just isn't there, and so on).
- As the HTML starts coming in, the browser begins parsing it right away, and picks up any links to CSS, JS, or images along the way and requests those too. Behind the scenes it builds a DOM tree from the HTML and a CSSOM from the styles, merges them into a render tree, and starts painting pixels on screen.

- And finally, JavaScript runs for anything interactive, form validation, small animations, pulling extra data from an API without needing to reload the whole page.

Q4. Tools Required to Set Up a Web Development Environment

Tool	Why it's needed
Code Editor (VS Code)	Where all the actual typing happens. Syntax highlighting, autocomplete and extensions make writing HTML/CSS/JS a lot less tedious.
A Web Browser (Chrome/Firefox)	For opening and testing the page and using DevTools when something looks off and needs debugging.
Git	Tracks every change made to the code, so nothing gets lost and makes it easy to undo something if it breaks.
GitHub (or GitLab)	Somewhere to push the code online. Works as a backup and makes sharing or collaborating easier.
Node.js + npm	Let's JavaScript run outside the browser; npm is how you install libraries instead of writing everything from scratch.
A local server (Live Server / XAMPP)	Serves the page locally, so it behaves like it's on a real host, including things like auto-refresh on save.
A database (MySQL / MongoDB)	Where actual data gets stored, user accounts, product listings, whatever the app needs to remember.
Browser DevTools	Built into the browser already; gets used constantly for inspecting elements, console errors, and network requests.
Build tools (Webpack / Vite)	Bundles the files together, optimizes them, and automates a lot of the repetitive setup work.
Postman	Good for testing backend APIs on their own, sending a request and checking the response before the frontend even exists.

Q5. What is a Web Server? Examples of Commonly Used Servers

A web server is software that just sits there waiting for requests from browsers. When one arrives, it fetches or builds whatever's being asked for (a page, an image, some bit of data) and sends it back. It might be running on an actual physical machine in a data centre, or on a cloud instance somewhere, but either way its job stays the same: listen, process, respond. Some web servers just hand out static files exactly as they are, while others work together with an application layer to put pages together dynamically.

A few that actually come up a lot in practice:

- Apache HTTP Server, which has been around for ages, is open-source, and is still used widely because of how configurable it is.
- Nginx, known for handling a huge number of connections without breaking a sweat; also, often used as a reverse proxy or load balancer sitting in front of other servers.
- Microsoft IIS, mostly seen on Windows Server setups and pairs naturally with ASP.NET applications.
- LiteSpeed, a commercial built to be a quicker drop-in replacement for Apache.
- Node.js with Express isn't a "traditional" server in the classic sense, but a handful of lines of JS code is enough to spin one up that handles requests directly.

Q6. Roles of a Frontend Developer, Backend Developer, and Database Administrator

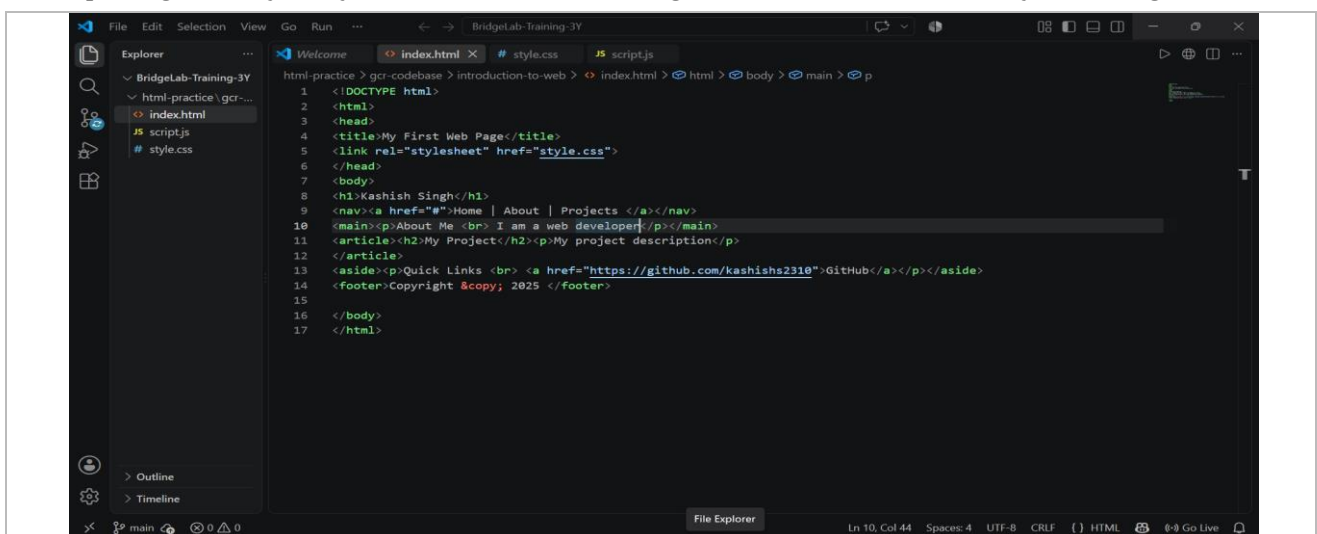
A front-end developer mainly worries about how the site looks and feels. Taking a design mockup and turning it into an actual, working, responsive page using HTML, CSS and JavaScript (and probably a framework like React), and making sure it doesn't fall apart on different screen sizes or browsers.

A backend developer handles everything happening behind that interface: server logic, building APIs, dealing with authentication and security, and making sure the app can talk properly to the database and any outside services it relies on.

A database administrator, or DBA, is more focused on the database itself. Designing the schema properly, taking regular backups, fixing slow queries, controlling who has access to what, and generally making sure the data stays correct and available when it's needed.

Q7. Install VS Code and Configure it for HTML, CSS, and JavaScript Development

- Grabbed VS Code from the official site, code.visualstudio.com, and picked up the installer for my OS.
- Ran the installer. On Windows it's worth ticking “Add to PATH” and “Add Open with Code action”, that way you can just right-click any folder later and open it straight in VS Code.
- Opened the Extensions panel (Ctrl+Shift+X) to add a couple of things I knew I'd need.
- Installed “Live Server” so previews auto-reload, “Prettier” to keep formatting consistent without thinking about it, and “ESLint” to catch JS mistakes as I go.
- Made a small project folder with index.html, style.css and script.js, and linked the CSS and JS inside the HTML file.
- Right-clicked index.html, chose “Open with Live Server”, and the page popped up in the browser, updating instantly every time I saved. Makes testing so much faster than manually refreshing.



Q8. Static vs Dynamic Websites

Aspect	Static Website	Dynamic Website
Content	Stays the same for every visitor unless it's manually edited	Changes depending on who's viewing it, the time, or what's stored in the database
Technology	Just HTML/CSS, maybe a little JS	HTML/CSS/JS plus a backend language and a database
Updating it	Someone has to edit the files and re-upload them	Updates itself automatically through the database or a CMS
Example	A simple portfolio page	An online store showing live prices and stock

Example: a college department's basic info page, faculty list, course names, contact details, the sort of thing that barely gets touched from month to month.

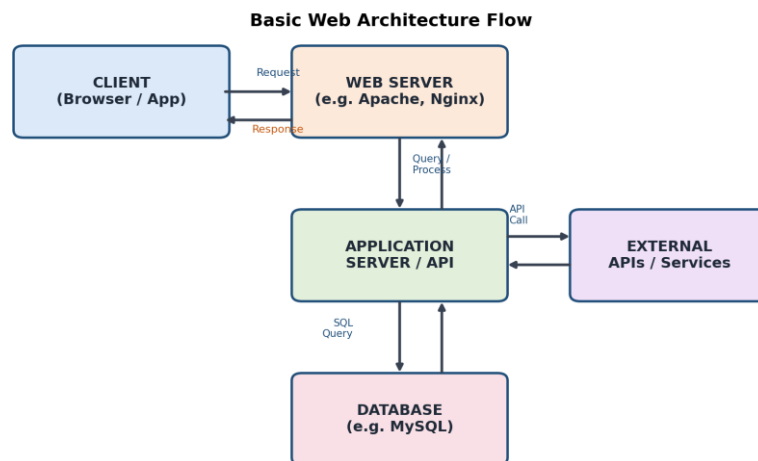
For dynamics, Instagram is a good one. Every time you open it the feed looks different because it's pulling fresh data (posts, likes, comments) from a database depending on who's logged in at that moment.

Q9. Five Web Browsers and Their Rendering Engines

Browser	Rendering Engine	Note
Google Chrome	Blink	Google's own engine, forked years back from WebKit. Still the one most people end up using.
Mozilla Firefox	Gecko	Mozilla's own engine, usually praised for sticking close to web standards and caring about privacy.
Safari	WebKit	Apple's engine, tuned mostly for performance and battery life on Mac and iOS.
Microsoft Edge	Blink	Used to run Microsoft's own EdgeHTML engine, but switched to Chromium's Blink back in 2020 for better compatibility with sites.
Opera	Blink	Chromium/Blink-based too, just with a few extras bundled in like a free VPN and an ad blocker.

The rendering engine is basically what decides how HTML and CSS actually get turned into pixels on your screen, which is why some sites look ever so slightly different depending on the browser. Chrome, Edge and Opera all share Blink, so they usually agree with each other, but Firefox (Gecko) and Safari (WebKit) can sometimes handle fonts, spacing, or newer CSS features a little differently. That's pretty much the reason for cross-browser testing is something developers actually have to think about.

Q10. Labeled Diagram: Basic Web Architecture Flow



The client sends a request over to the web server, which hands it off to the application or API layer. That layer runs a query against the database whenever it needs data, and it might also reach out to some external API if the app depends on an outside service. Once everything it needs has come back, the data flows back up through the application server and web server, and eventually lands back at the client as a response the browser turns into a page.